

QRepo Design Draft

Quantii Repo | Working design | 26 September 2026

QRepo is a proposed versioned, synchronized project workspace. It keeps ordinary file editing and explicit commits, while making synchronization, historical browsing, automatic checkpoints, and programmable merge policies part of the repository itself. This draft records the ideas discussed so far; commands and data structures below are illustrative where their exact syntax has not been decided.

Product model

A repository has a current project state, an immutable history of named checkpoints, and storage for file content. Devices may synchronize changes to the current state without treating each edit as a commit. A commit records an intentional checkpoint of a complete included project state, with a message and optional tags or other metadata. The working directory remains the ordinary editable view of the present.

Layer	Responsibility
Live state	Hold local edits and reconcile concurrent changes across devices or collaborators. CRDT operations may be useful for suitable text and structured data.
Checkpoint history	Record stable project snapshots and metadata. Sync activity between checkpoints does not create user-visible commits.
Object storage	Store content efficiently, including binary assets, through immutable objects, deduplication, and optional remote replication.

New commits from others synchronize automatically during typical connected usage, applying merge functions where needed. When reconciliation succeeds, this requires no manual pull or merge step. Synchronizing uncommitted live edits is a separate policy decision. Commit and push are distinct operations internally; `qr` commit performs both by default.

Working directory and commands

The repository root includes all files and directories by default. Explicit ignore rules exclude generated output, caches, secrets, or other paths. Repository metadata lives under `.qrepo`; a project-level ignore file or equivalent policy supplies the exclusions. There is no staging area: commit takes the current included project state as a batch.

```
qr init
qr ignore build/
qr status
qr commit "Add Unicode identifier parsing"
qr history
qr diff
qr tag parser-v2
```

A command named `qr` is the current leading choice; QRepo or Quantii Repo names the system. Names, syntax, and the behavior of tag and timestamp flags are provisional. Recorded timestamps should come from the commit event, while user-supplied time annotations, if supported, should be distinguishable.

Commit and offline behavior

A commit records the included project snapshot as an entry in the local log. By default, `qr` commit also attempts to push and synchronize, applying configured merge functions when histories diverge. Offline, the local entry is still created immediately and synchronization remains pending. Connectivity is not a prerequisite for recording local history.

Result	Meaning
Synced	Checkpoint is durable locally and acknowledged by the configured remote or peers under the repository durability policy.
Local pending	Commit is recorded in the local log; push and reconciliation are pending, for example while offline.
Conflict	Local history remains recorded, but synchronization needs a resolution. Further commits on the affected branch are blocked until resolved.

When connectivity returns, pending local commits and incoming commits synchronize automatically. A conflict pauses reconciliation without undoing an already recorded local entry. Status must distinguish local recording from remote acknowledgement. Separate commit-only and push operations may be exposed; their syntax and acknowledgement timing remain open.

Automatic commits and programmable policy

Autocommit is a first-class setting. A trigger function can inspect elapsed time and a change threshold; a commit builder can supply a default timestamped message or be replaced with a project-specific function that adds summaries, tags, or metadata. An LLM summary is optional policy, not an implicit prerequisite for creating a checkpoint.

```
autocommit.trigger = elapsed >= 20 minutes || changed_lines >= 200
autocommit.message = "Autocommit: " + timestamp()
```

These expressions describe the proposed policy surface, not a settled configuration language. Thresholds should be evaluated against a defined baseline, and autocommit must obey the same conflict, durability, and validation rules as a manual commit.

Merge model

A merge strategy receives a common base and two concurrent states, and either produces a candidate result or reports a conflict. Identical or unchanged inputs resolve directly. Disjoint changes can combine automatically; strictly speaking, fast forward describes an ancestry case, while combining two independent edits is a merge. Rules may dispatch by file type, path, and project-defined behavior.

```
merge(base, left, right) -> Merged(candidate, evidence) | Conflict(details)

*.json  -> structural merge
*.md    -> Markdown-aware merge
src/**  -> syntax-aware merge -> optional LLM candidate -> manual
```

For JSON, changes to distinct keys can usually combine, provided the result parses and satisfies any schema. A Markdown strategy might combine nonoverlapping sections, but blindly concatenating overlapping prose can duplicate or contradict text. Text, AST, and project-specific strategies can form an ordered fallback chain.

Validation and provenance

A strategy proposes a candidate; validators determine whether it is acceptable. Examples include parse, schema, typecheck, build, and tests. These checks cannot prove that a merge preserved intent, especially for LLM-generated changes, so policy may require manual review. A recorded merge should identify the strategy, inputs, validators run, outcomes, and any human approval.

Unresolved conflicts

When rules cannot settle a conflict, the affected branch enters a conflict state. Editing and inspection remain possible, but further commits on that branch are blocked until resolution and validation complete. Existing local log entries remain available. Commands such as `qr resolve` and `qr resolved` could open a resolver and record the intentional resolution. The internal representation of a resolution remains to be specified.

History and editor experience

The editor can open immutable historical project views alongside current files without changing the working directory or entering detached HEAD. It can show the checkpoint, author, message, comments, and earlier versions relevant to a line or symbol. Restoring a historical file creates a change in the present rather than moving the current view backward.

```
qr history
qr restore <checkpoint> src/parser.q
qrepo://project/commit/<checkpoint>/src/parser.q
```

Comments and commit notes should have stable anchors that survive ordinary edits where possible. Historical browsing needs a clear read-only indicator so users do not mistake an old version for the present file.

Proposed checkpoint record

```
Commit {
  id, root_state, parents, author, timestamp, message, tags, metadata
}
```

A root state identifies a complete immutable project snapshot. Parent links retain ancestry for concurrent offline work and reconciliation, even if most users never operate the graph directly. Content addressing and structural sharing can make full snapshots efficient without making the storage representation part of the everyday interface.

Decisions to make next

- Decide whether uncommitted edits also synchronize continuously, beyond the automatic synchronization of commits; define how these edits interact with incoming snapshots.
- Specify remote authority, acknowledgement timing, and how reconciliation is recorded alongside existing local commits. Decide whether to expose commit-only and push commands.
- Choose CRDT coverage by data type. Arbitrary binary files, renames, deletes, and directory moves need explicit concurrent-change rules.
- Specify a safe merge policy for each type, including when an automatic result requires human review and which validators are mandatory.
- Define branch or line semantics, access control, and whether tags move or remain immutable.
- Choose the repository policy language, execution sandbox, determinism rules, and how policy versions are recorded alongside decisions.